Prelectura: Fundamentos de R para lingüística de corpus

Prelectura: Fundamentos de R para lingüística de corpus

Prelectura: Fundamentos de R para lingüística de corpus

Módulo de Análisis Exploratorio de Datos, Doctorado en Lingüística. Basado en Gries, ***Quantitative Corpus Linguistics with R*** (Routledge, 2009), capítulo 3.

Las sesiones del módulo están dedicadas al análisis de datos, no a la sintaxis de R. Esta lectura cubre el mínimo necesario para seguirlas. No hace falta memorizar nada: hace falta haber ejecutado el código y resuelto las actividades. Instala primero R (cran.r-project.org) y luego RStudio (posit.co/download/rstudio-desktop). Escribe los ejemplos a mano en un guion de código (***script***) propio; en los bloques de código, las líneas que comienzan con #> muestran la salida (***output***) que R devuelve y no deben escribirse.

1. Nociones centrales

R evalúa expresiones y guarda resultados mediante asignación con `<-`. Los nombres distinguen mayúsculas de minúsculas (***case-sensitive***): `corpus` y `Corpus` son objetos distintos.

```
frecuencia <- 42
frecuencia
#> [1] 42
```

Los tipos de datos (***data types***) básicos son `numeric` (números), `character` (texto, siempre entre comillas) y `logical` (`TRUE` o `FALSE`, en mayúsculas y sin comillas). La función `class()` indica el tipo de un objeto. Sin comillas, R interpreta el texto como el nombre de un objeto y falla si no existe.

Las funciones reciben argumentos (***arguments***), por posición o por nombre, y muchos tienen un valor predeterminado (***default***): `round(3.14159, digits = 2)` devuelve 3.14; `round(3.14159)` devuelve 3, porque `digits` vale 0 de forma predeterminada. La ayuda de cualquier función se consulta con `?round`; la sección ***Arguments*** de la página de ayuda es la que más se usa.

2. Vectores

Un vector es una secuencia de elementos del mismo tipo y se crea con `c()`. Si se mezclan tipos, R convierte todo al más general sin avisar: `c(1, 2, "tres")` produce un vector de texto. Este fenómeno, la coerción (***coercion***), es fuente frecuente de errores silenciosos. Un caso útil: `TRUE` vale `1` y `FALSE` vale `0`, así que `sum()` sobre un vector lógico cuenta cuántos elementos cumplen una condición.

Los elementos pueden llevar nombre, lo que resulta natural para tablas de frecuencia:

```
frec <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)
```

El acceso a los elementos se llama indexación (***indexing***) y se hace con corchetes. R indexa desde `1`. Se puede indexar por posición (`frec[3]`), por rango (`frec[2:4]`), por exclusión (`frec[-1]`) o por nombre (`frec["que"]`).

La forma más importante es la indexación lógica (***logical indexing***): se pasa una condición y R devuelve los elementos que la cumplen.

```
frec[frec > 2000]
#>   el   de  que
```

3. Factores

Un factor es la estructura de R para variables categóricas. Sus categorías posibles son los niveles (***levels***), que de forma predeterminada se ordenan alfabéticamente. Este es el punto donde la teoría de escalas de medición se vuelve código: un factor sin orden es una variable nominal; uno declarado con `ordered = TRUE` es ordinal.

```
pos <- factor(c("NOUN", "VERB", "ADJ", "NOUN", "VERB", "NOUN"))
table(pos)
#>  ADJ NOUN VERB
#>    1    3    2
```

`table()` produce la tabla de frecuencias y, a diferencia de lo que ocurre con texto simple, conserva los niveles con frecuencia cero tras un filtrado; `droplevels()` los elimina si es necesario. El orden de los niveles se controla al crear el factor con el argumento `levels`, lo que determina el orden de las categorías en tablas y gráficos.

Para escalas ordinales se declara el orden explícitamente:

```
acept <- factor(c("marginal", "aceptable", "inaceptable"),
                levels = c("inaceptable", "marginal", "aceptable"),
```

4. Marcos de datos (***data frames***)

Un marco de datos es una tabla: cada columna es un vector, todas las columnas tienen la misma longitud, y columnas distintas pueden ser de tipos distintos. Es el formato habitual para metadatos de corpus.

```
meta <- data.frame(
  texto  = c("T01", "T02", "T03", "T04", "T05"),
  genero = c("narrativo", "académico", "narrativo", "periodístico", "académi
  tokens = c(1250, 3400, 890, 2100, 4200),
  types  = c(520, 980, 410, 760, 1150)
)
```

Al abrir cualquier conjunto de datos, ejecuta `str()`: muestra dimensiones, nombres, tipos y una muestra de valores. Complementan `dim()`, `head()` y `summary()`.

Una columna se extrae con `meta$tokens` y es un vector ordinario, de modo que todo lo de la sección 2 aplica. La indexación bidimensional usa `[fila, columna]`, con la coma obligatoria: `meta[1, ]` es la primera fila y `meta[, "tokens"]` la columna completa.

5. Listas

Una lista puede contener objetos de cualquier tipo y longitud, incluidas otras listas o marcos de datos completos. Su relevancia práctica es doble.

Primero, los resultados de las pruebas estadísticas son listas, y saber recorrerlas da acceso a componentes que la salida no muestra:

```
resultado <- t.test(x, y)
names(resultado)
resultado$p.value
```

Segundo, cuando una función propia debe devolver varias cosas, la lista es la solución natural.

La distinción central: el corchete simple `[ ]` devuelve una sublista; el doble `[[ ]]` devuelve el contenido del elemento. La notación `$nombre` equivale al doble corchete. Como los elementos son objetos completos, la indexación se encadena: `analisis$frecuencias["que"]`.

Para aplicar una función a cada elemento sin escribir bucles se usan `lapply()`

6. Programación elemental

Un condicional `if` ejecuta código cuando se cumple una condición, con alternativas mediante `else` y `else if`. Evalúa un solo valor: sobre vectores produce un error. Para condiciones elemento a elemento se usa `ifelse(condición, valor_si, valor_no)`, la forma habitual de crear variables categóricas a partir de numéricas. Con más de dos categorías, `cut()` divide un vector numérico en intervalos etiquetados y devuelve directamente un factor.

Un bucle (***loop***) `for` repite un bloque para cada elemento de un vector:

```
for (p in c("casa", "sol")) {
  cat(p, "tiene", nchar(p), "caracteres\n")
}
```

Cuando el bucle produce resultados, crea el contenedor antes (`res <- numeric(n)`) y recorre con `seq_along()`. Pero la regla general es otra: si existe una versión vectorizada, úsala. El ejemplo anterior se resuelve mejor con `nchar(palabras)` directamente. Reserva los bucles para procesos que dependen del paso anterior o que leen varios archivos. El bucle `while` repite mientras una condición sea verdadera y se

Referencias

Gries, S. Th. (2009). ***Quantitative Corpus Linguistics with R***. Routledge. Capítulo 3.
Wickham, H. y Grolemund, G. (2017). ***R for Data Science***. O'Reilly. Disponible en r4ds.hadley.nz. Levshina, N. (2015). ***How to Do Linguistics with R***. John Benjamins.